

Zuverlässig laden – Reactive Systems im Einsatz für Elektromobilität

BOB 2026

inoio

inoio – unsere Reise

INTERSPORT®

HAMBURGER
ENERGIENETZE

OTTO

HOCHBAHN

Tomorrow

BITMARCK®

inoio (zu dritt) gegründet

Zufriedenheitsmaximierung
statt Gewinnmaximierung

Fridays for Future
GWÖ Bilanz

Verantwortungseigentum

MA GbR

2011

2019

'20

'23

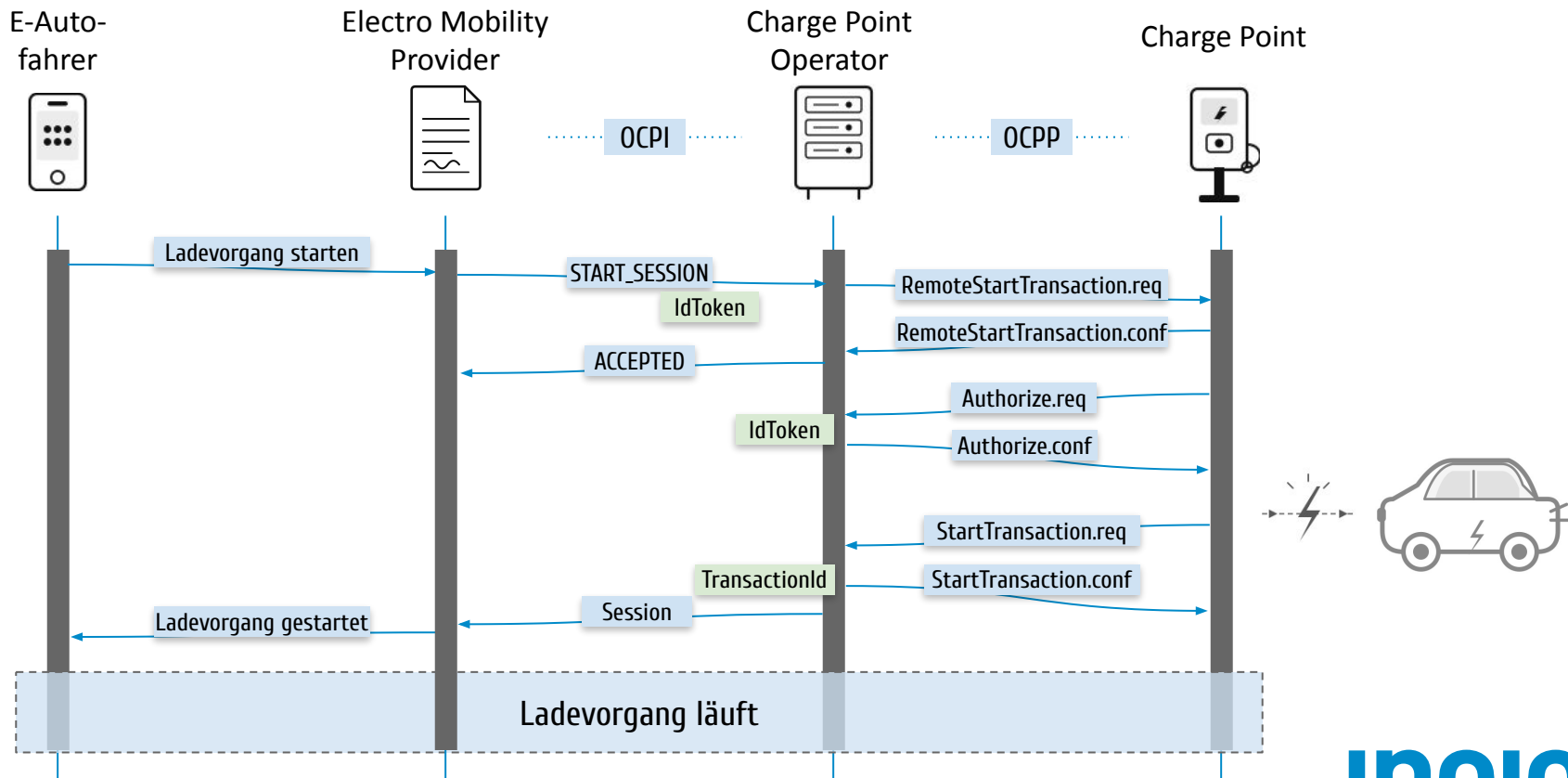
'26

inoio

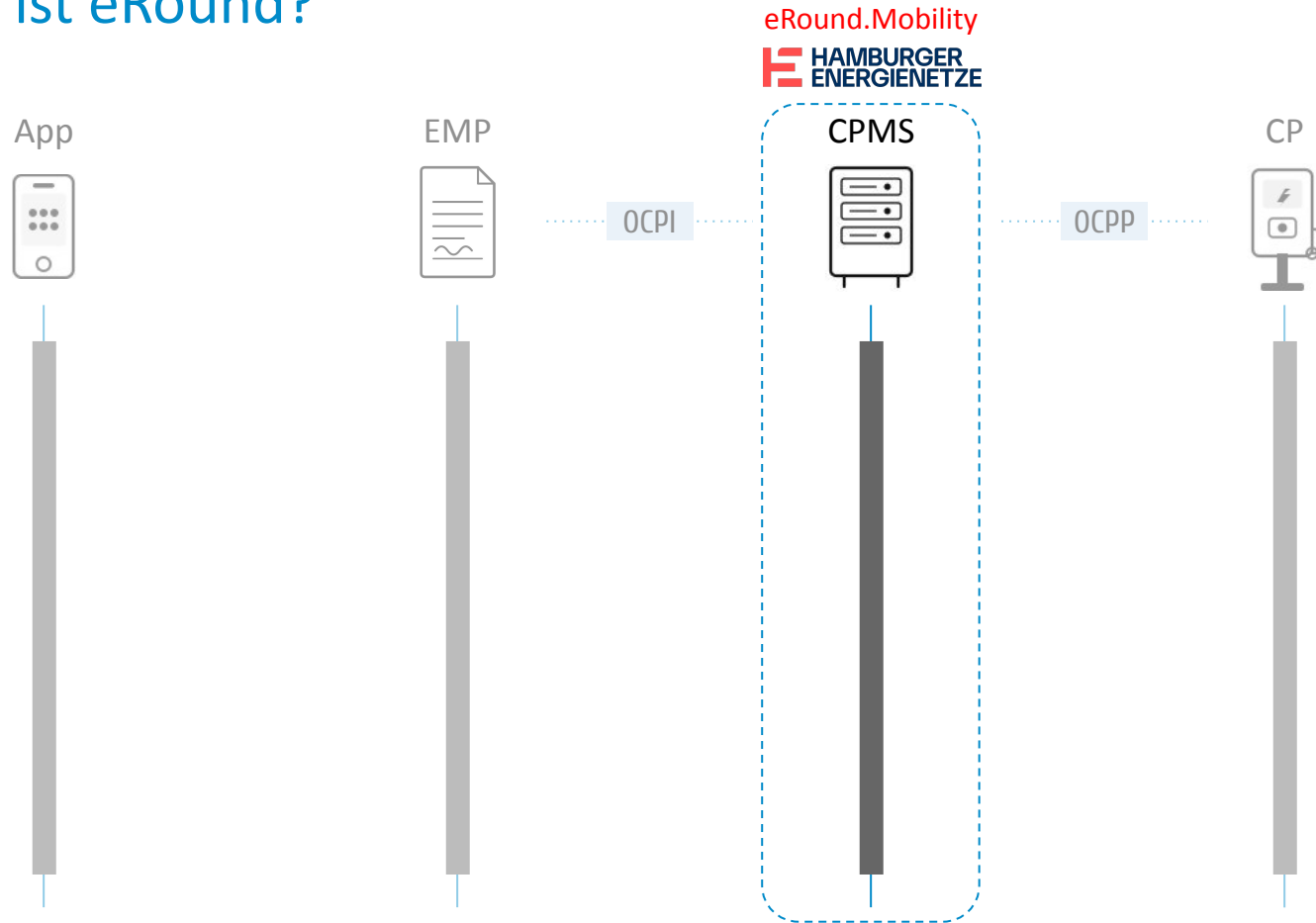
Reactive eMobility Systems - Motivation

- Relevanz: **Nutzerakzeptanz und Ladeerlebnis sind wichtig für Mobilitätswende**
 - Was ist Mobilitätswende und wozu brauchen wir sie?
 - Akzeptanz – Erwartung: Laden → Bezahlen → Fertig.
 - Realität: verteiltes System mit Fehlern!
- Leitfrage: **Wie baut man ein System (zur Steuerung von Ladeinfrastruktur), das zuverlässig funktioniert, auch wenn Netzwerk, Hardware und externe Partner es nicht sind?**

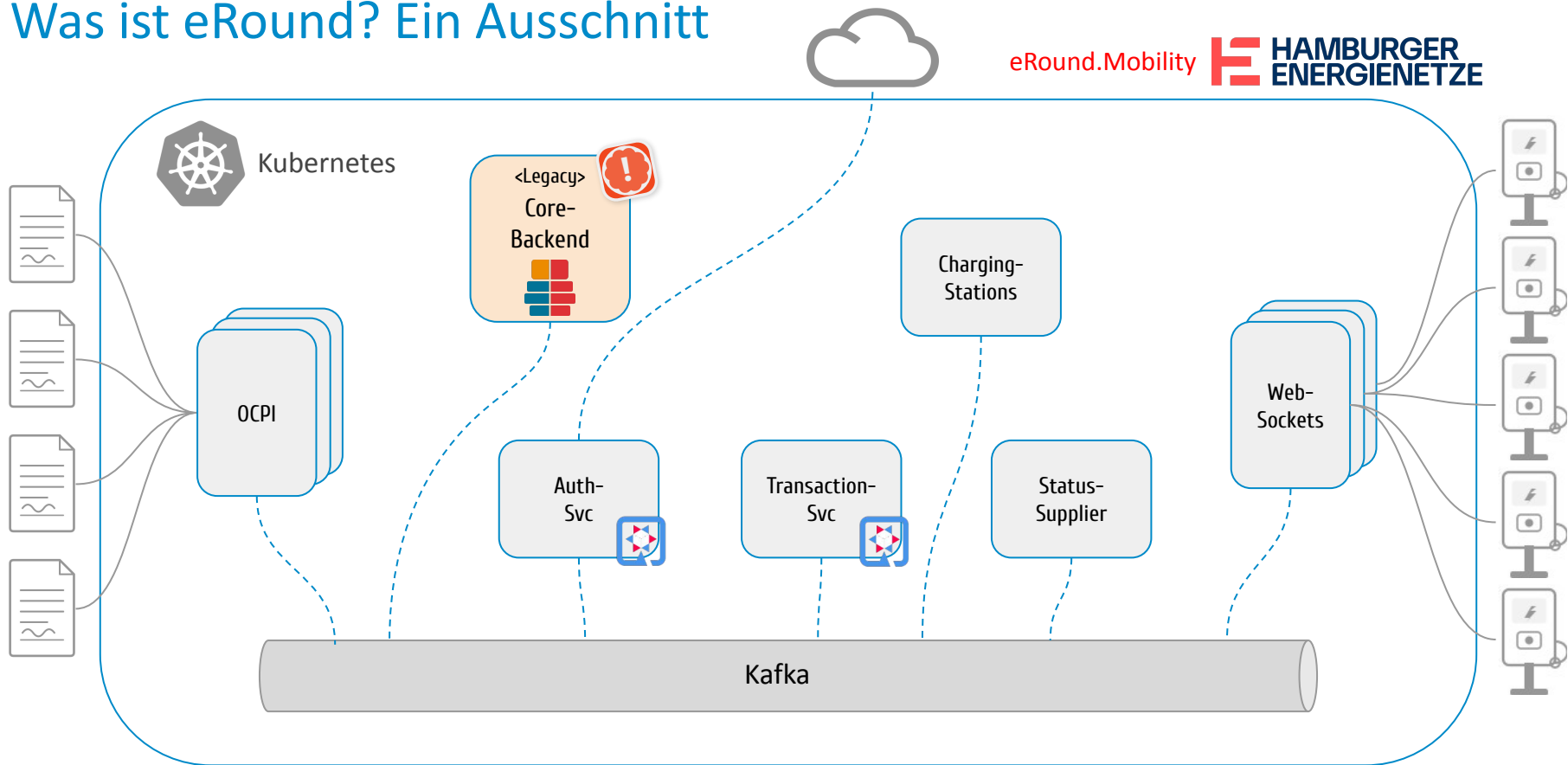
Der Ladevorgang – technisch betrachtet



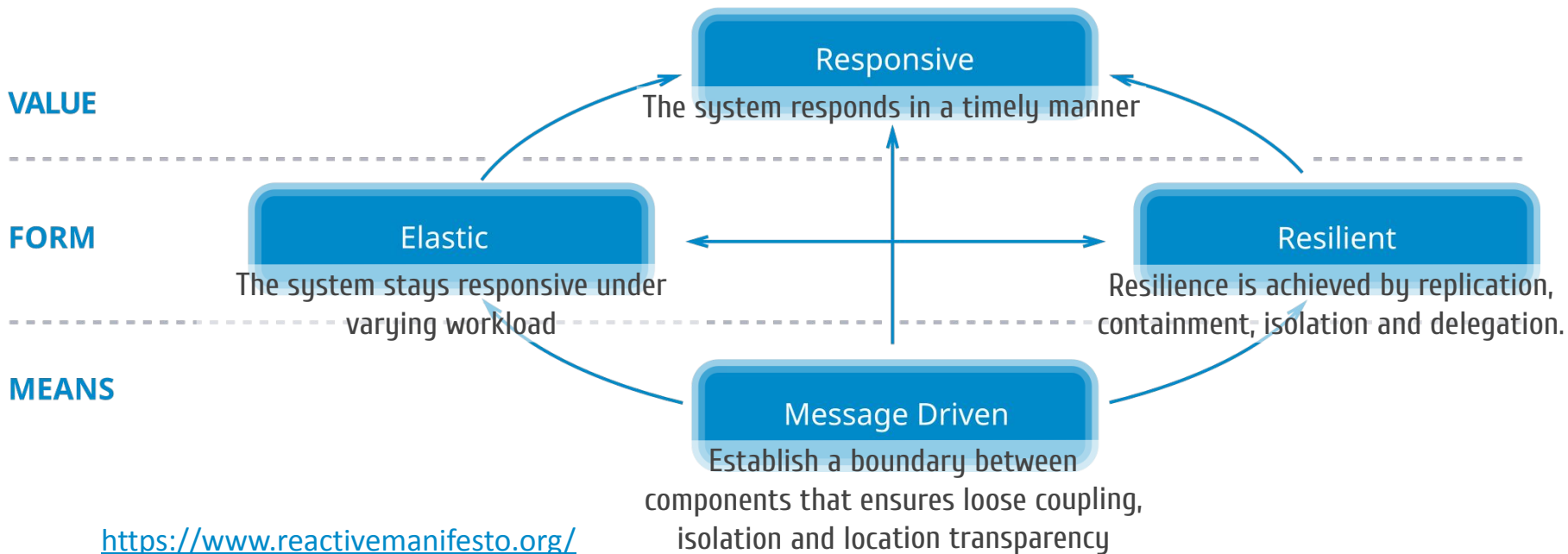
Was ist eRound?



Was ist eRound? Ein Ausschnitt



Zuverlässigkeit & Reactive Manifesto in der Praxis

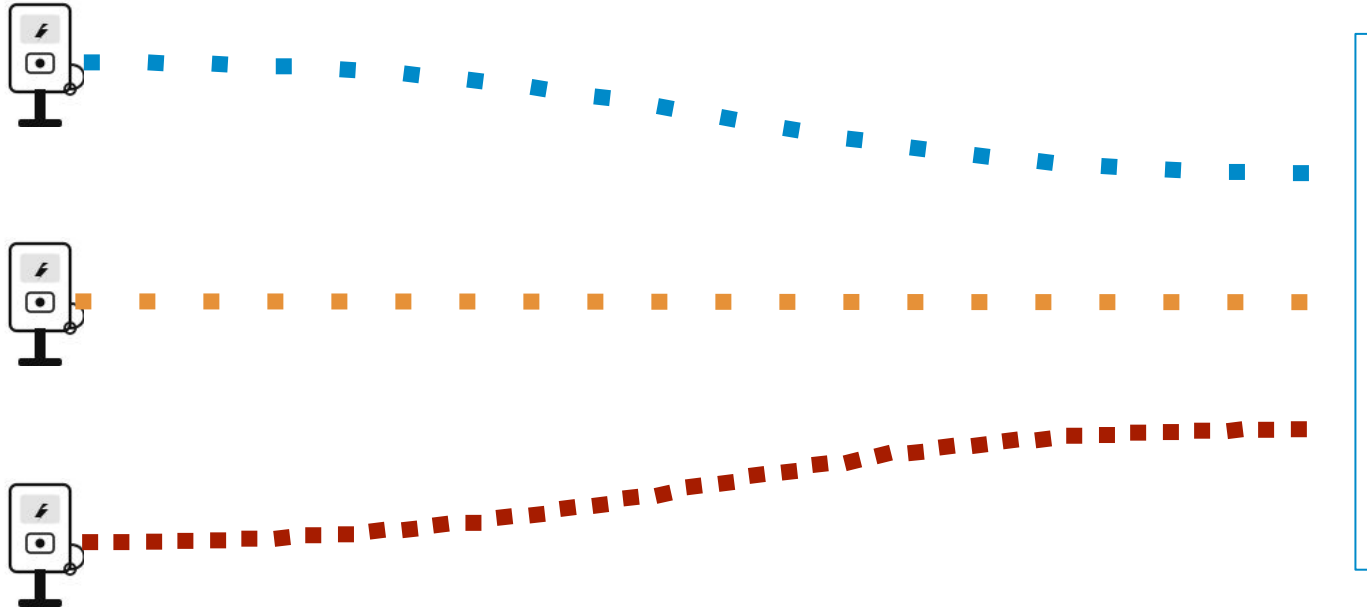


Reactive Systems – Lösungen

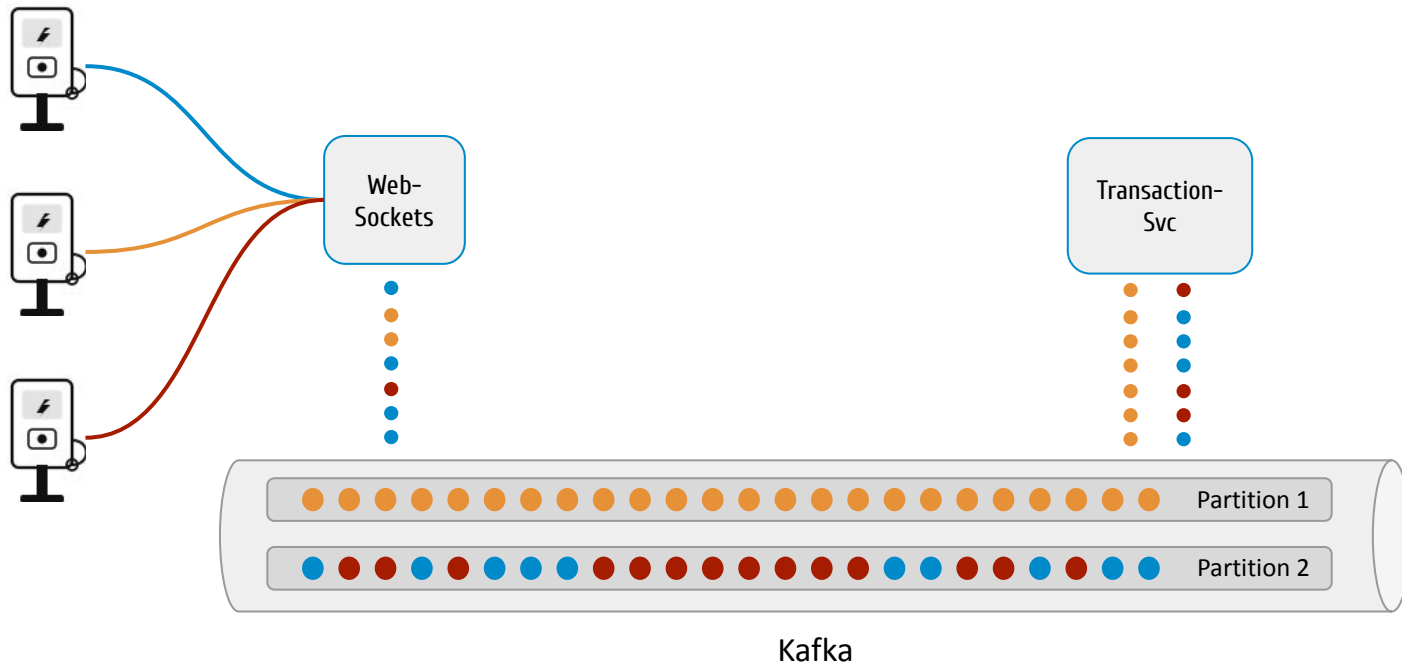
Übliche Lösungen

- Lose gekoppelte Services (Microservices / Self-Contained Systems)
 - Laufen repliziert, mit mehreren Instanzen
- Event-getriebene Architektur, asynchrone Integration der Services (benötigte Daten werden repliziert)
 - Aufteilen der Datenmengen (Partitionierung/Sharding)
- Kubernetes als Container Runtime
- Resilience-Patterns: z. B. Retries, Rate-Limiting etc.

Challenge: Geordnete Streams pro Ladesäule



Challenge: Geordnete Streams pro Ladesäule



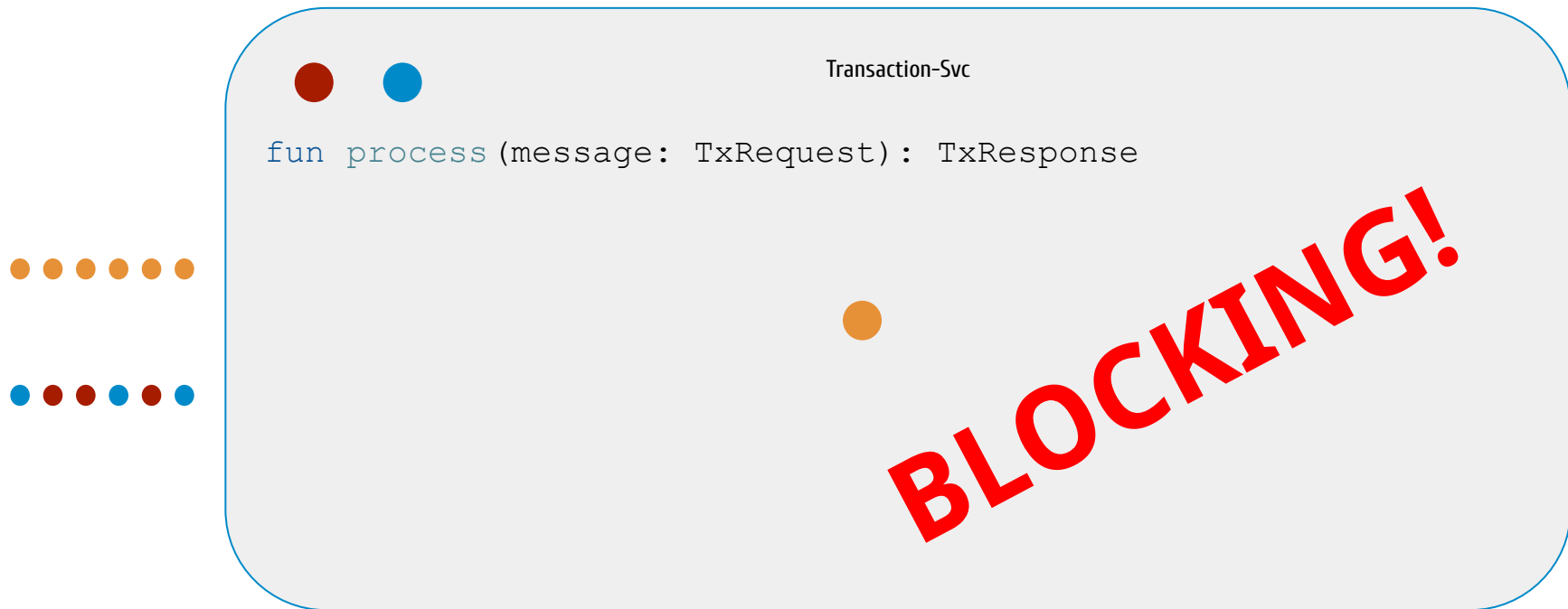
Challenge: Geordnete Streams pro Ladesäule

Transaction-Svc

```
fun process (message: TxRequest) : TxResponse
```



Challenge: Geordnete Streams pro Ladesäule



Challenge: Geordnete Streams pro Ladesäule



The diagram shows a light gray rounded rectangle representing a window titled "Transaction-Svc". At the top left are two window control buttons, a red one and a blue one. Below them is a code snippet in a monospaced font: `fun process (message: TxRequest) : TxResponse`. An orange dot is positioned below the word `process`. To the left of the window, there are two horizontal rows of dots: the top row has six orange dots and the bottom row has six dots in an alternating blue and red pattern. Inside the window, there is a dashed rectangular box containing the text "Was wollen wir?" followed by a bulleted list of four items.

Transaction-Svc

```
fun process (message: TxRequest) : TxResponse
```

Was wollen wir?

- Parallele Verarbeitung für unterschiedliche LS
- Reihenfolge pro Ladesäule
- At-least-once Delivery
- Elastizität

Easy?! Geordnete Streams pro Ladesäule

Was wollen wir?

- Parallele Verarbeitung
- Reihenfolge pro Ladesäule
- At-least-once Delivery
- Elastizität

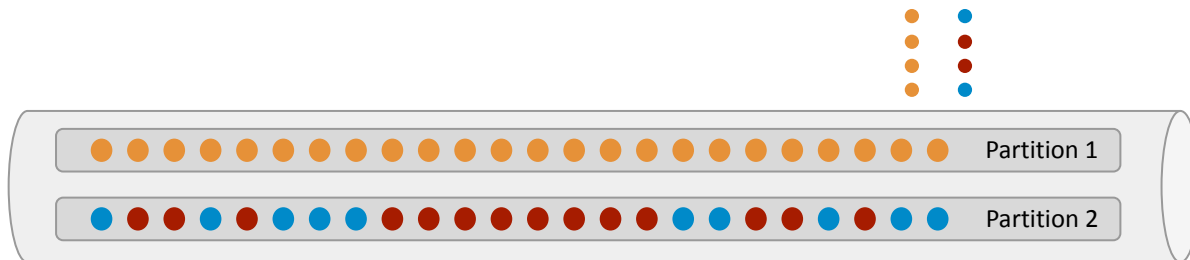
Das bauen wir selbst!

- Ein Virtual Thread pro Ladesäule
- KafkaListener erhält Nachrichten und übergibt sie an den jeweiligen Thread

Transaction-Svc

```
val threadsByCsId = Map<CsId, VirtualThread>()

@KafkaListener
fun process(msg: Message): Unit {
    threadsByCsId[msg.csId].accept(msg)
}
```



Easy?! Geordnete Streams pro Ladesäule

Was wollen wir?

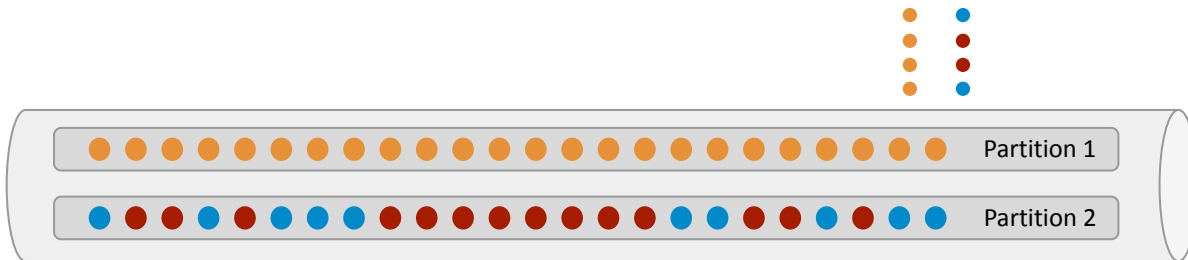
- Parallele Verarbeitung
- Reihenfolge pro Ladesäule
- At-least-once Delivery
- Elastizität

Das bauen wir selbst!

- Der Thread puffert die Nachricht
- Nachricht für Nachricht wird verarbeitet
- Jede verarbeitete Nachricht wird bestätigt

Transaction-Svc

```
class VirtualThread {  
  val msgs = Queue<Message>()  
  
  fun accept(msg: Message): Unit = msgs.put(msg)  
  
  fun run(): Unit = while(true) {  
    msgs.take()?.let { msg ->  
      process(msg)  
      msg.acknowledge()  
    }  
  }  
}
```



Easy?! Geordnete Streams pro Ladesäule

Was wollen wir?

- Parallele Verarbeitung
- Reihenfolge pro Ladesäule
- At-least-once Delivery
- Elastizität

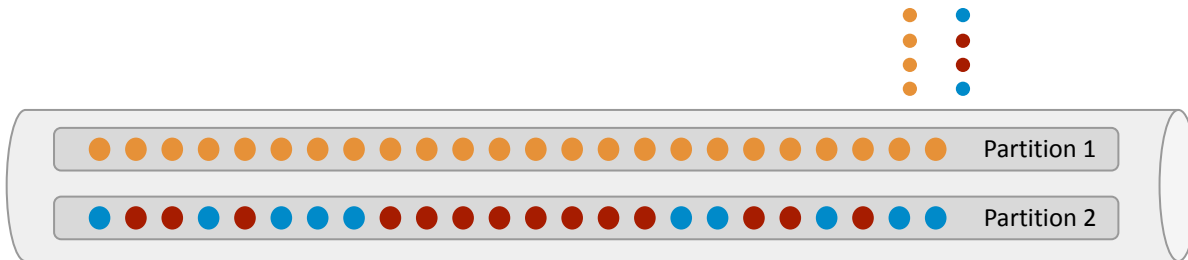
Das bauen wir selbst!

- Der Thread puffert die Nachricht
- Nachricht für Nachricht wird verarbeitet
- Jede verarbeitete Nachricht wird bestätigt

Transaction-Svc

```
class VirtualThread {  
    val msgs = Queue<Message>()  
  
    fun accept(msg: Message): Unit = msgs.put(msg)  
  
    fun run(): Unit = while(true) {  
        msgs.take()?.let { msg ->  
            process(msg)  
            msg.acknowledge()  
        }  
    }  
}
```

Fertig?!



Easy?! Geordnete Streams pro Ladesäule

Was wollen wir?

- Parallele Verarbeitung
- Reihenfolge pro Ladesäule
- At least once Delivery
- Elastizität

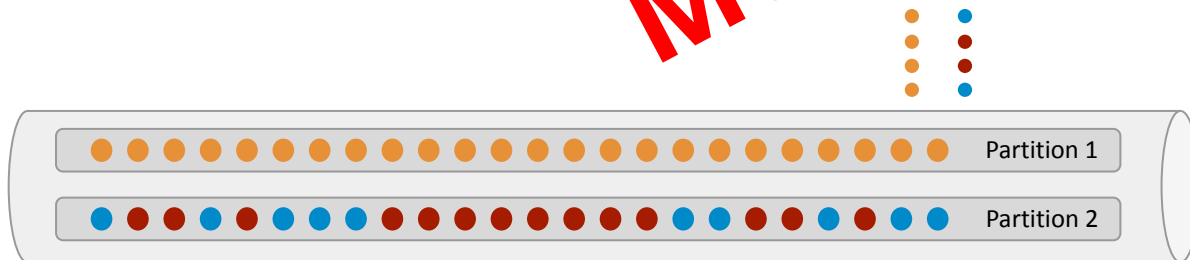
Das bauen wir selbst!

- Der Thread puffert die Nachricht
- Nachricht für Nachricht wird verarbeitet
- Jede verarbeitete Nachricht wird bestätigt

Transaction-Svc

```
class VirtualThread {  
    val msgs = Queue<Message>()  
  
    fun accept(msg: Message): Unit = msgs.put(msg)  
  
    fun run(): Unit = while(true) {  
        msgs.take()?.let { msg ->  
            process(msg)  
            msg.acknowledge()  
        }  
    }  
}
```

Message Loss!



Easy?! Geordnete Streams pro Ladesäule

Was wollen wir?

- Parallele Verarbeitung
- Reihenfolge pro Ladesäule
- ~~At least once Delivery~~
- Elastizität

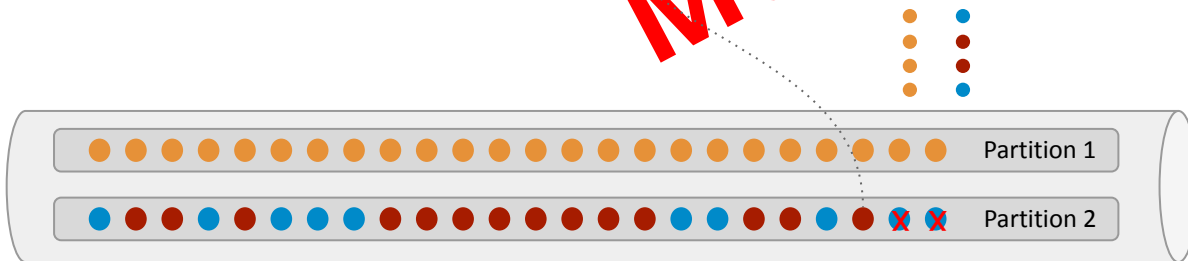
Das bauen wir selbst!

- Der Thread puffert die Nachricht
- Nachricht für Nachricht wird verarbeitet
- Jede verarbeitete Nachricht wird bestätigt

Transaction-Svc

```
class VirtualThread {  
    val msgs = Queue<Message>()  
  
    fun accept(msg: Message): Unit = msgs.put(msg)  
  
    fun run(): Unit = while(true) {  
        msgs.take()?.let { msg ->  
            process(msg)  
            msg.acknowledge()  
        }  
    }  
}
```

Message Loss!



Easy?! Geordnete Streams pro Ladesäule

Herausforderungen

- Was bestätigen wir?

Was wollen wir?

- Parallele Verarbeitung
- Reihenfolge pro Ladesäule
- At-least-once Delivery
- Elastizität

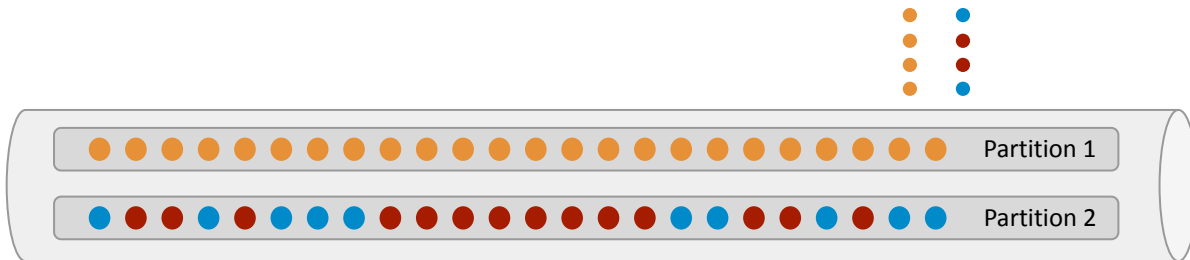
Transaction-Svc

```
@KafkaListener
fun process(msg: Message): Unit =
    threadsByCsId[msg.csId].accept(msg)

class VirtualThread {
    val msgs = Queue<Message>(42)

    fun accept(msg: Message): Unit = msgs.put(msg)

    fun run(): Unit = while(true) {
        msgs.take()?.let { msg ->
            process(msg)
            msg.acknowledge()
        }
    }
}
```



Easy?! Geordnete Streams pro Ladesäule

Was wollen wir?

- Parallele Verarbeitung
- Reihenfolge pro Ladesäule
- At-least-once Delivery
- Elastizität

Herausforderungen

- Was bestätigen wir?
- Wann laden wir wieviel?

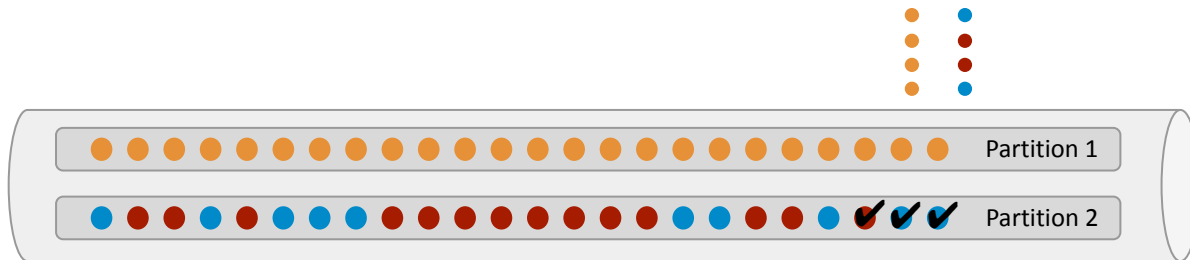
Transaction-Svc

```
@KafkaListener
fun process(msg: Message): Unit =
    threadsByCsId[msg.csId].accept(msg)

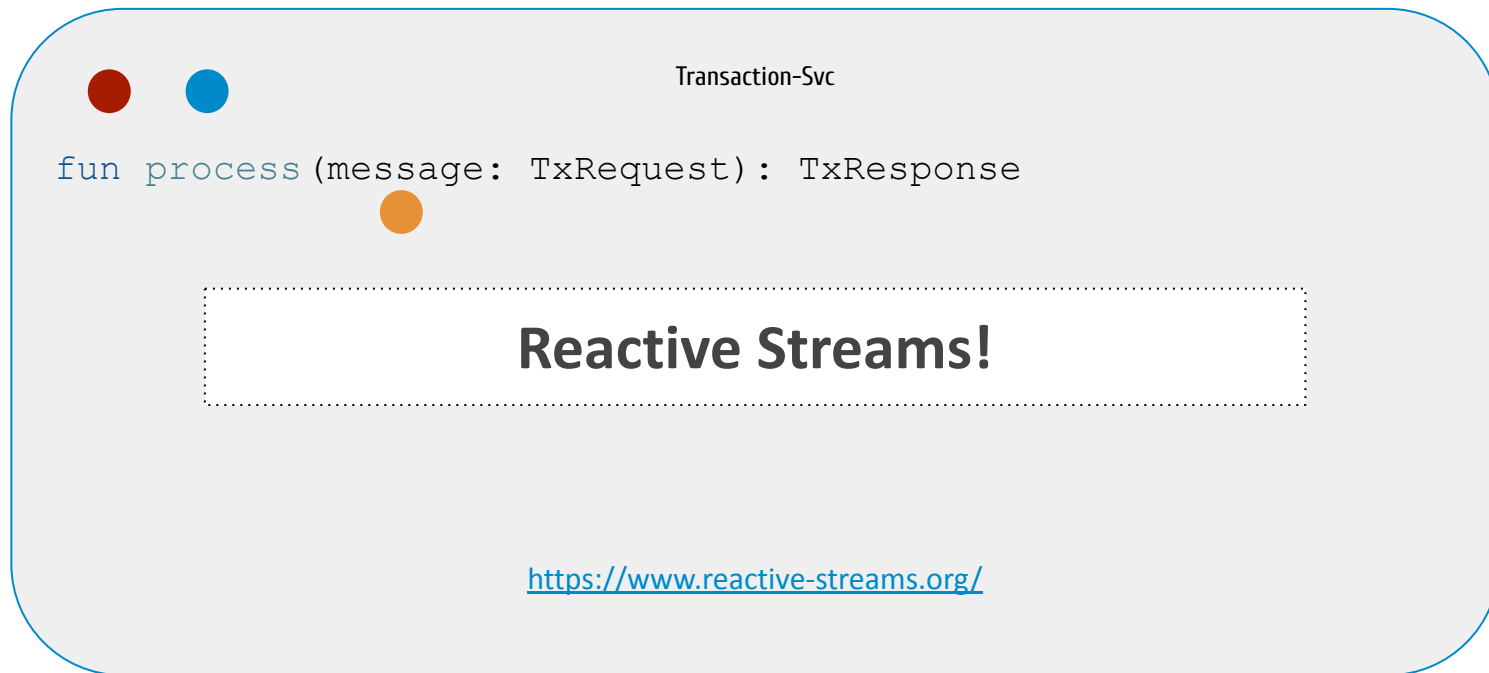
class VirtualThread {
    val msgs = Queue<Message>(42)

    fun accept(msg: Message): Unit = msgs.put(msg)

    fun run(): Unit = while(true) {
        msgs.take()?.let { msg ->
            process(msg)
            msg.acknowledge()
        }
    }
}
```



Again: Geordnete Streams pro Ladesäule



The diagram shows a light gray rounded rectangle representing a window titled "Transaction-Svc". At the top left are two colored circles (red and blue). Below them is a code editor with the text `fun process (message: TxRequest) : TxResponse`. An orange circle is positioned below the code. In the center is a white rounded rectangle with a dotted border containing the text "Reactive Streams!". At the bottom is a blue hyperlink <https://www.reactive-streams.org/>. To the left of the window are two horizontal rows of colored dots: the top row has six orange dots and the bottom row has six dots in alternating blue and red colors.

Transaction-Svc

```
fun process (message: TxRequest) : TxResponse
```

Reactive Streams!

<https://www.reactive-streams.org/>

Reactive: Geordnete Streams pro Ladesäule

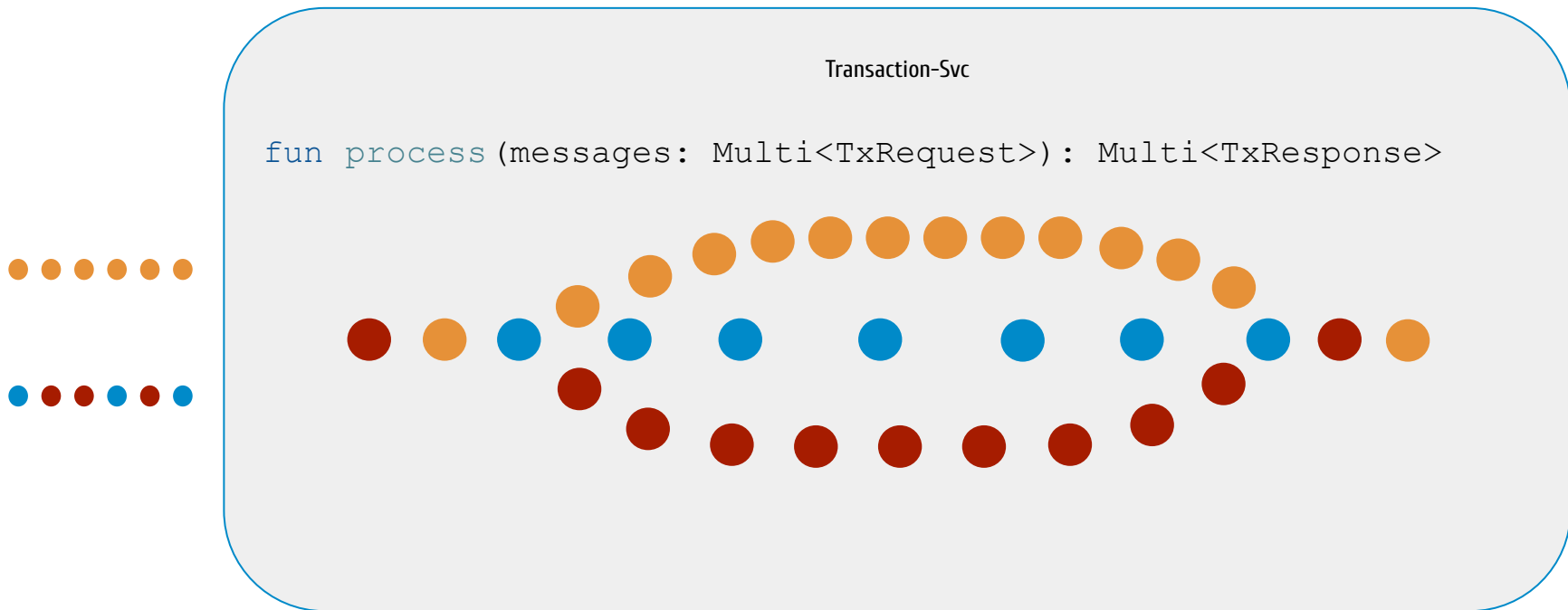
Transaction-Svc

```
fun process(messages: Multi<TxRequest>): Multi<TxResponse>
```



<https://quarkus.io/guides/mutiny-primer>

Reactive: Geordnete Streams pro Ladesäule



Reactive: Geordnete Streams pro Ladesäule

Transaction-Svc

```
fun process(messages: Multi<TxRequest>): Multi<TxResponse> =  
    messages.group().by { it.csId }
```



Reactive: Geordnete Streams pro Ladesäule

Transaction-Svc

```
fun process(messages: Multi<TxRequest>): Multi<TxResponse> =  
    messages.group().by { it.csId }  
        .map { groupedMulti ->  
            DefaultKeyedMulti(groupedMulti.key(), groupedMulti)  
        } // Multi<KeyedMulti<CsId, TxRequest>>
```



Reactive: Geordnete Streams pro Ladesäule

Transaction-Svc

```
fun process(messages: Multi<TxRequest>): Multi<TxResponse> =  
    messages.group().by { it.csId }  
        .map { groupedMulti ->  
            DefaultKeyedMulti(groupedMulti.key(), groupedMulti)  
        } // Multi<KeyedMulti<CsId, TxRequest>>  
        .flatMap { keyedMsgs ->  
            process(keyedMsgs)  
        }
```



Reactive: Geordnete Streams pro Ladesäule

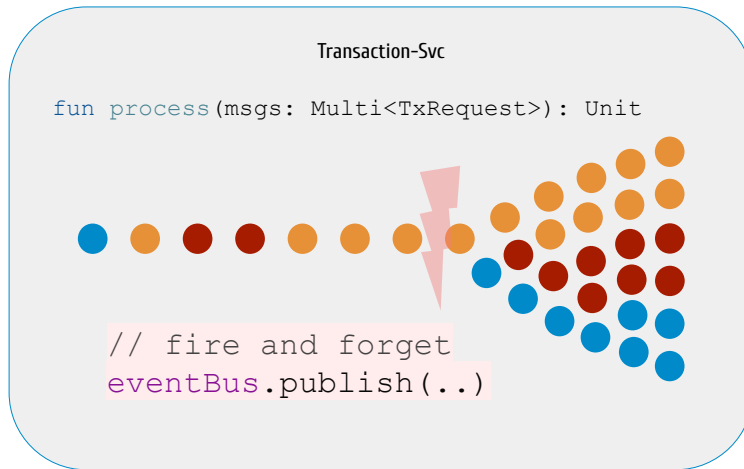
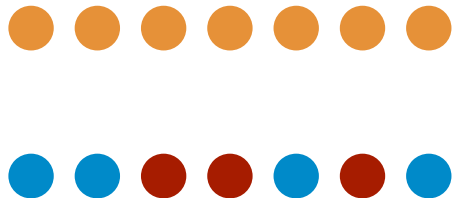
Transaction-Svc



```
fun process(messages: Multi<TxRequest>): Multi<TxResponse> =  
    messages.group().by { it.csId }  
        .map { groupedMulti ->  
            DefaultKeyedMulti(groupedMulti.key(), groupedMulti)  
        } // Multi<KeyedMulti<CsId, TxRequest>>  
        .flatMap { keyedMsgs ->  
            process(keyedMsgs)  
        }
```

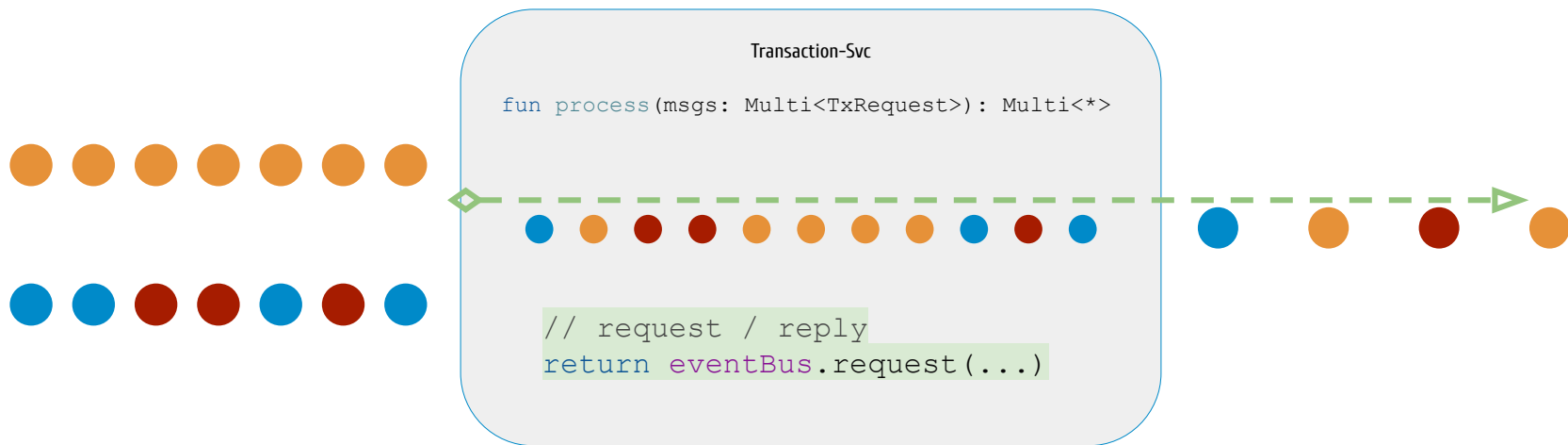
```
private fun process(keyedMsgs: KeyedMulti<CsId, TxRequest>):  
Multi<TxResponse>
```

Reactive? Langsamer Producer/Output



<https://quarkus.io/guides/vertx-reference#eventbus>

Elastic & Backpressure



<https://quarkus.io/guides/vertx-reference#eventbus>

Elastische, isolierte Streams pro Ladesäule

Was haben wir?

- ✓ Parallele Verarbeitung
- ✓ Reihenfolge pro Ladesäule
- ✓ At-least-once Delivery
- ✓ Elastizität

Transaction-Svc

```
fun process(messages: Multi<TxRequest>): Multi<TxResponse> =  
    messages.group().by { it.csId }  
        .map { groupedMulti  
            DefaultKeyedMulti(groupedMulti.key(), groupedMulti)  
            // Multi<KeyedMulti<CsId, TxRequest>>  
        }.flatMap { keyedMsgs ->  
            process(keyedMsgs)  
        }
```

```
private fun process(keyedMsgs: KeyedMulti<CsId, TxRequest>):  
    Multi<TxResponse>
```

Fertig.

Was nehmen wir mit?

- Entkopplung über asynchrone Verarbeitung ist (hier) notwendig für Responsiveness

Was nehmen wir mit?

- Entkopplung über asynchrone Verarbeitung ist (hier) notwendig für Responsiveness
- Asynchrone Verarbeitung bringt neue Herausforderungen mit sich

Was nehmen wir mit?

- Entkopplung über asynchrone Verarbeitung ist (hier) notwendig für Responsiveness
- Asynchrone Verarbeitung bringt neue Herausforderungen mit sich
- Reactive Streams bieten eine Lösung auf Applikationsebene, für Isolation, Elasticity und Responsiveness

Was nehmen wir mit?

- Entkopplung über asynchrone Verarbeitung ist (hier) notwendig für Responsiveness
- Asynchrone Verarbeitung bringt neue Herausforderungen mit sich
- Reactive Streams bieten eine Lösung auf Applikationsebene, für Isolation, Elasticity und Responsiveness
- Reaktiv + funktional spielen gut zusammen

Was nehmen wir mit?

- Entkopplung über asynchrone Verarbeitung ist (hier) notwendig für Responsiveness
- Asynchrone Verarbeitung bringt neue Herausforderungen mit sich
- Reactive Streams bieten eine Lösung auf Applikationsebene, für Isolation, Elasticity und Responsiveness
- Reaktiv + funktional spielen gut zusammen
- Herausforderungen sind domänenspezifisch – Reactive Systems können zur erfolgreichen Mobilitätswende beitragen

Was nehmen wir mit?

- Entkopplung über asynchrone Verarbeitung ist (hier) notwendig für Responsiveness
- Asynchrone Verarbeitung bringt neue Herausforderungen mit sich
- Reactive Streams bieten eine Lösung auf Applikationsebene, für Isolation, Elasticity und Responsiveness
- Reaktiv + funktional spielen gut zusammen
- Herausforderungen sind domänenspezifisch – Reactive Systems können zur erfolgreichen Mobilitätswende beitragen
- Übertragbarkeit z.B.: IoT (Messstellenbetrieb), Logistik, Zahlungsverkehr

Referenzen

1. Akzeptanz E-Mobilität: <https://epb.bibl.th-koeln.de/frontdoor/index/index/year/2023/docId/2404>
2. Reactive Manifesto: <https://www.reactivemanifesto.org/>
3. Reactive Streams: <https://www.reactive-streams.org/>
4. Mutiny: <https://quarkus.io/guides/mutiny-primer>
5. Quarkus Vert.x Event Bus: <https://quarkus.io/guides/vertx-reference#eventbus>

Q & A

Quarkus (Smallrye) Kafka Connector

Die default Commit Strategie (“throttled”) macht genau was wir wollen:

» ***throttled** keeps track of received messages and commits an offset of the **latest acked message in sequence** (meaning, all previous messages were also acked). This strategy **guarantees at-least-once delivery even if the channel performs asynchronous processing.**«*

<https://quarkus.io/guides/kafka#commit-strategies>